

Chapter 3 - Context Passing and Session Management

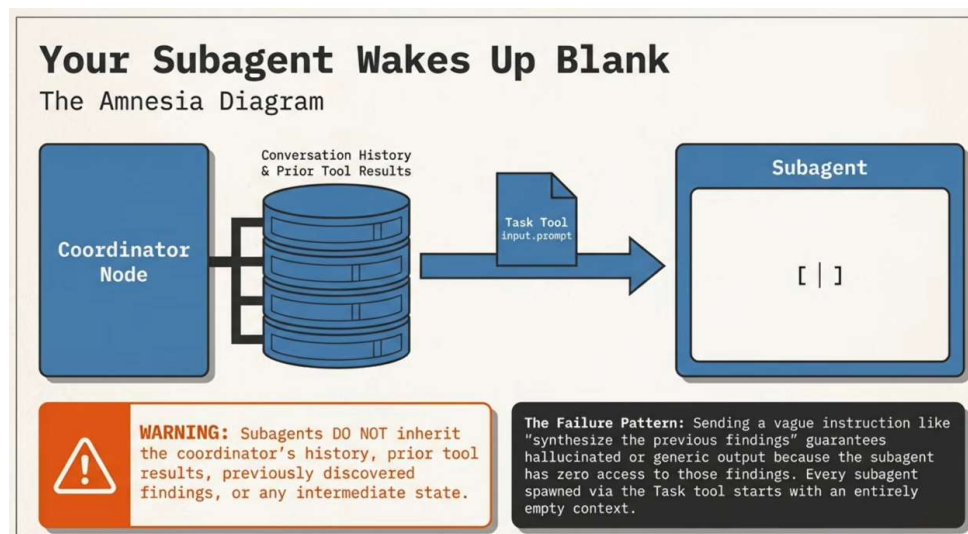
06 September 2026 12:51

How Agents work inside a project | How to use coordinator | Session managements and Subagents

Scenario -

- Massive research project | Team of Analysts | I am the coordinator and let's say analysts are subagents

If we ask to pass on Sub-Agent1 findings to Sub-Agent2 then 1 won't understand what to pass context to 2. We already know coordinator is supposed to pass context because by default individual subagents starts with scratch.



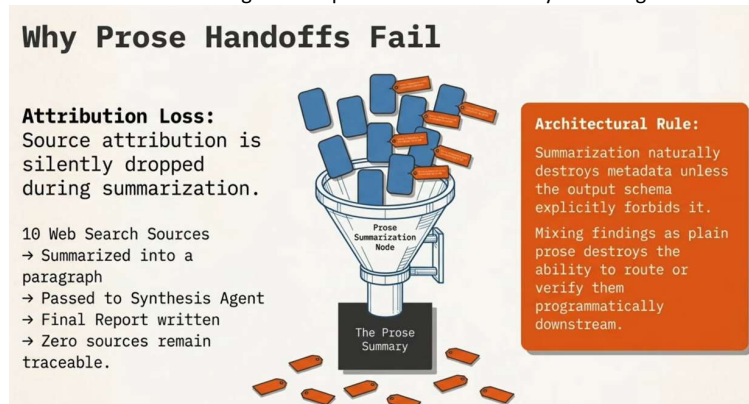
Subagents do not inherit coordinator conversation history. Let's say coordinator is running for 10 minutes, tons of context accumulated. Let's say it (coordinator) invokes a subagent. Coordinator passes only to subagent2 what it needs to know.

So, here in task_tool call via which coordinator is invoking subagent, we will pass on some context as well. All boils down to good prompt engineering. Reason every subagent starts from scratch and reason why coordinator does not pass anything to the subagent is to prevent context contamination and context dilution.

Let's look at a good prompt and bad prompt -

Bad prompt -

Here are some recent findings from different researches, help me prepare a summary. => In this case subagent will hallucinate or produce a generic report. It doesn't know exactly what all work was done by other agents. **Coordinator knows this though.** This is a classic failure in multi agent architecture. Hence we need to understand how to pass the context properly. So, we need to do structured context prompting. | Either hallucinate or provide a generic report. | Maybe the results of the individual agents are passed on to the final synthesis agent but still coordinator is not aware of it yet.



Solution: **Structured context finding**

Good prompt example -

< check video 08:00 for example >

This time what we do is using the same old task_tool_call() we pass a synthesis agent which contains all the findings from individual agents. We call it synthesis agent. We are taking help of coordinator to build context for synthesis agent.

```
research_findings = [
    {
        "finding": "Claude's tool_use stop_reason requires the full assistant message to be appended before tool results.",
        "source_url": "https://docs.anthropic.com/agents/tool-use",
        "source_title": "Anthropic Tool Use Documentation",
        "page_number": None,
        "retrieved_at": "2025-03-15T09:12:00Z",
        "confidence": "high"
    },
    {
        "finding": "Using an iteration cap as a primary stop condition is an anti-pattern.",
        "source_url": "https://docs.anthropic.com/agents/loops",
        "source_title": "Agentic Loop Best Practices",
        "page_number": None,
        "retrieved_at": "2025-03-15T09:14:00Z",
        "confidence": "high"
    }
]
```

```
# — Build the synthesis prompt with full context —
synthesis_prompt = f"""
You are a synthesis agent. Your task is to produce a structured research report.

FINDINGS TO SYNTHESIZE:
{json.dumps(research_findings, indent=2)}

Requirements:
- Every claim in the report MUST cite its source_url
- Preserve the retrieved_at date for temporal accuracy
- Flag any conflicting findings with both sources annotated
- Output format: JSON with keys: summary, claims[], conflicts[]
"""
```

```
# — Pass to subagent via Task tool —
task_call = {
    "type": "tool_use",
    "name": "Task",
    "input": {
        "description": "Synthesize research findings into structured report",
        "prompt": synthesis_prompt
    }
}
```

Common production issue -

When info is summarized to a synthesis agent then we loss info on which info came from which agent. We do not know if hallucination happened or not. So, attribution loss takes place. Source attribution is silently dropped during summarization. To fix this - we use **"Claim source Mappings"**

Instead of passing summary as plain text, we pass them as array where every claim is passed with its source.

```
intermediate_output = {
    "claims": [
        {
            "claim_id": "c001",
            "text": "Subagents do not inherit coordinator conversation history.",
            "source_id": "src_001",  # links back to source object
            "confidence": "high"
        },
        {
            "claim_id": "c002",
            "text": "Parallel subagents require multiple Task calls in one coordinator response.",
            "source_id": "src_002",
            "confidence": "high"
        }
    ],
    "sources": [
        {
            "source_id": "src_001",
            "url": "https://docs.anthropic.com/multi-agent",
            "title": "Multi-Agent Architecture Guide",
            "retrieved_at": "2025-03-15T09:12:00Z"
        },
        {
            "source_id": "src_002",
            "url": "https://docs.anthropic.com/task-tool",
            "title": "Task Tool Documentation",
            "retrieved_at": "2025-03-15T09:14:00Z"
        }
    ]
}
```

```

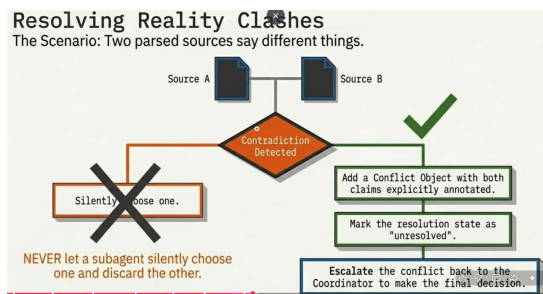
subagent_context.py > ...
60     },
61     ],
62     "sources": [
63         {
64             "source_id": "src_001",
65             "url": "https://docs.anthropic.com/multi-agent",
66             "title": "Multi-Agent Architecture Guide",
67             "retrieved_at": "2025-03-15T09:12:00Z"
68         },
69         {
70             "source_id": "src_002",
71             "url": "https://docs.anthropic.com/task-tool",
72             "title": "Task Tool Reference",
73             "retrieved_at": "2025-03-15T09:14:00Z"
74         }
75     ],
76     "conflicts": [] # if two sources say different things, both go here
77 }
78
79 # — When passing to the next agent, pass the WHOLE object —
80 # Never extract just "claims" and drop "sources"
81 # The next agent needs both to produce accurate output
82
83

```

➤ Coming back to the slide -

"Resolving Reality Clashes -

What if two different sources are talking about the same thing but they are giving different results. What to do then?



If multiple conflicts happen in an agentic workflow then eventually coordinator should decide which one to move ahead with.

Create a conflict object and mention 2 different claims with their sources.

```

# — When two sources conflict, add to conflicts array —
conflict = {
    "conflict_id": "conf_001",
    "topic": "parallel subagent invocation",
    "claim_a": {
        "text": "Multiple Task calls in one response run in parallel.",
        "source_id": "src_002"
    },
    "claim_b": {
        "text": "Task calls are always sequential regardless of placement.",
        "source_id": "src_003"
    },
    "resolution": "unresolved" # coordinator must decide
}

```

We pass it on to coordinator and let it take the decision. Hence, conflict scenario in an multi agentic flow pipeline, always annotate both with the attribution and pass it down to the coordinator.

Never let a subagent silently resolve a conflict. This is a direct exam question.

New topic: **Managing State**

➤ Whenever we are working with Claude code in an agentic context we are not always starting with scratch.

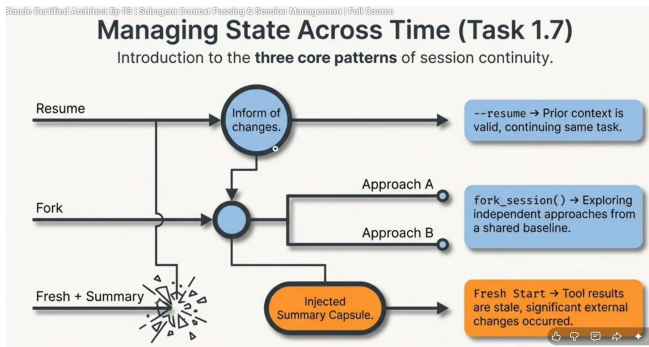
Normal Claude code scenario : Claude always remembers what happens before. But if claude suddenly loses context, then we have ways to preserve context/session.

We do it via 3 mechanisms -

1. Named session resumption - Most straight forward | Start claude session with a name , next time when you trigger claude, it will remember that session via the name.
e.g., we usually trigger claude by saying "Claude"
Now we can say "claude --session-name "session_name_my_choice"

Next time when we start claude we can say "claude --resume "session_name"

Note: Claude --resume also works but it will resume most recent session.



Note: While defining resume session I the code, we should make sure that we are updating resume session from our previous analysis so that it remembers what we did in the last session. e.g., if while working with claude agent, if some fiels were deleted and after that we added some files, the agent needs to remember that.

```

98
99 # -- Best practice: inform resumed session about changes --
100 resume_context = """
101 Resuming from previous analysis session.
102
103 CHANGES SINCE LAST SESSION:
104 - auth/middleware.py has been modified (new rate limiting added)
105 - requirements.txt updated (added redis==5.0.1)
106 - The TODO in payment_service.py line 142 has NOT been addressed yet
107
108 Please re-analyze only the changed files and continue from your prior findings.
109 """

```

Exam note: Never resume the session without explicitly stating what was changed in the last session

2. Fork session (2nd method) : This is like github branching but here it happens in context of agents.
e.g., we have a baseline analysis that took a long time to build up. The agent knows a lot of things via that work. What if we want to explore another approach independent from the shared baseline. We try another approach called as a forked session.

Example code below where we have created 2 forks from the same shared repo -

```

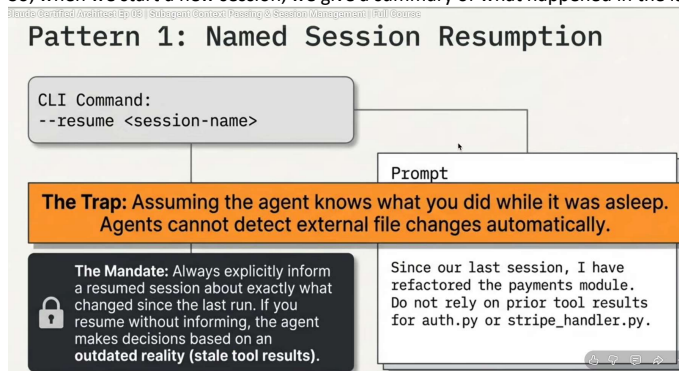
1 # Fork session (2nd method)
2
3 import anthropic
4
5 client = anthropic.Anthropic()
6
7 # -- Baseline session with full codebase analysis --
8 baseline_session_id = "baseline-analysis-481"
9
10 # Request to explore new approach for refactoring
11
12 fork_a = client.beta.sessions.fork(
13     session_id=baseline_session_id,
14     system_prompt_addition="Explore refactoring approach A: extract service layer pattern."
15 )
16
17 fork_b = client.beta.sessions.fork(
18     session_id=baseline_session_id,
19     system_prompt_addition="Explore refactoring approach B: CQRS pattern."
20 )
21
22 # -- Run both forks independently --
23 # fork_a gets the same baseline context but explores approach A
24 # fork_b gets the same baseline context but explores approach B
25 # Neither contaminates the other

```

3. Fresh + Summary = Sometimes we may want to start a new session rather than resuming th old session. We need to take that call.
We resume when prior context is valid. However, **we start fresh when our tool_results are stale or when extended context degrades model reasoning capability.**

What if 80% of codebase changed as per our last session, then would it still make sense to resume the last session ? In those cases we should rather start fresh session.
Exam note: **When should we resume vs start fresh??** => We take this decision based on the scenario. If it is valid then we resume otherwise we create a new session.

So, when we start a new session, we give a summary of what happened in the last session. That way we won't be starting from fresh altogether.



Cloud Certified Architect (CC) | Subagent Context, Prompting & Session Management | Full Course

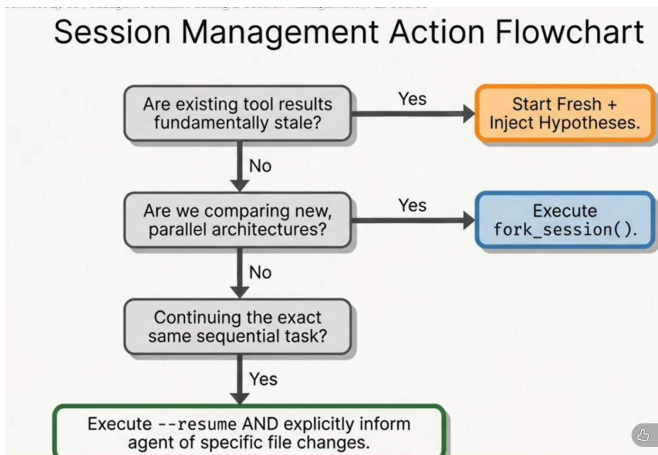
Pattern 3: Fresh Start + Injection

The Trigger:
When tool results are fundamentally stale, resuming is actively counterproductive. Start fresh, but compress prior findings into an injected summary in the initial prompt.

Critical Framing Rule:
The prompt must frame the injected summary as 'hypotheses to validate, not established facts.' This psychological cue prevents the model from blindly accepting information that may have changed.

The Session Routing Diagnostic

Current Context State	Codebase Volatility	Required Action Command
Context is Valid / Shared	Zero / Low Volatility	Use <code>--resume</code> + explicitly state updates.
Need Parallel Exploration	Shared Baseline context	Use <code>fork_session()</code> .
Tool Results are Stale	High Volatility / Time Passed	Use Fresh Session + Inject structured hypotheses.



Anti pattern means we should not do these things. So listed below are those anti patterns.

Architectural Anti-Patterns (Exam Constraints)

❌ NOT THAT: Passing <i>prose summaries</i> between agents (silently drops attribution).	✅ DO THIS: Pass structured context via <i>Finding Objects</i> .
❌ NOT THAT: Letting a subagent <i>silently resolve source conflicts</i> (destroys provenance).	✅ DO THIS: Annotate both, mark "unresolved," escalate to Coordinator.
❌ NOT THAT: Using <i>fork_session</i> to handle targeted file updates.	✅ DO THIS: Fork is for parallel exploration only.
❌ NOT THAT: Resuming a session with a <i>heavily changed</i> codebase.	✅ DO THIS: Start fresh, inject hypotheses.
❌ NOT THAT: Resuming without <i>informing</i> the agent of changes.	✅ DO THIS: Explicitly declare updated reality.

Scenario question -

Sample Scenario Breakdown

Scenario 3 (Multi-Agent Research)	Scenario 2 (Code Generation)
The Bug:	The Bug:

Sample Scenario Breakdown

Scenario 3 (Multi-Agent Research)

The Bug:

Claim attribution is lost because web results are summarized into prose before synthesis.

The Fix:

Require structured objects with explicit `claim_id` → `source_id` mappings. Require the synthesis output schema to preserve them.

Scenario 2 (Code Generation)

The Bug:

Codebase analysis ran 3 days ago. Payments module was just refactored.

The Fix:

Resume via `--resume`. Explicitly inform the agent which specific files changed to trigger targeted re-analysis. Avoid a full fresh start to save prior baseline work.

Since it was just a refactor, we can access the session using session name. However, if let's say code was changed entirely (80%) then we can say that yes start a new session and give summary of last session

Exam-Critical Facts Summary

Task 1.3: Subagent Invocation

- > Context is empty by default.
- > Pass findings as structured objects.
- > Require `source_url`, `retrieved_at`, `confidence`.
- > Preserve `claim-source` mapping arrays.
- > Unresolved conflicts strictly escalate to Coordinator.

Task 1.7: Session State

- > `--resume` requires explicit manual context updates.
- > `fork_session()` is for parallel exploration only.
- > Stale tool results mandate a Fresh Session.
- > Injected prior findings must be framed as hypotheses.